Almacenamiento y Procesamiento de Grandes Datos

Subtaller 2: Repaso de SQL con Python

En este taller usaremos `sqlite3` (incluido en Python) para practicar conceptos fundamentales de SQL sobre una base de datos de e-commerce.

Modelo de datos

```
customers     products      orders          order_items
----------    ----------    ----------      -----------
customer_id   product_id    order_id        item_id
name          name          customer_id  ->  order_id
email         category      order_date      product_id
city          price         status          quantity
country       stock                         unit_price
```

Contenido

1. Configuración de la base de datos
2. SELECT básico y filtros (WHERE)
3. Ordenamiento (ORDER BY) y límites (LIMIT)
4. Funciones de agregación (COUNT, SUM, AVG, MAX, MIN)
5. Agrupación (GROUP BY) y filtros de grupo (HAVING)
6. JOIN entre tablas
7. Subconsultas
8. Funciones de texto y fecha
9. Ejercicios integradores

```
import sys
import pandas as pd

print('Dependencias listas para Colab.')
print(f'  - sqlite3: incluido en Python {sys.version.split()[0]}')
print(f'  - pandas {pd.__version__}')
```

```
Dependencias listas para Colab.
  - sqlite3: incluido en Python 3.12.13
  - pandas 2.2.2
```

1. Configuración: Crear la base de datos

```
import sqlite3
import pandas as pd

# Crear conexión en memoria (o cambiar ':memory:' por 'ecommerce.db' para persistir)
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Función auxiliar para mostrar resultados como DataFrame
def query(sql, params=None):
    if params:
        return pd.read_sql_query(sql, conn, params=params)
    return pd.read_sql_query(sql, conn)

print('Conexión establecida exitosamente.')
```

```
Conexión establecida exitosamente.
```

```
# Crear tablas
cursor.executescript('''
CREATE TABLE customers (
```

```
    customer_id TEXT PRIMARY KEY,
    name        TEXT NOT NULL,
    email       TEXT UNIQUE,
    city        TEXT,
    country     TEXT DEFAULT 'Colombia'
);

CREATE TABLE products (
    product_id  TEXT PRIMARY KEY,
    name        TEXT NOT NULL,
    category    TEXT,
    price       REAL NOT NULL,
    stock       INTEGER DEFAULT 0
);

CREATE TABLE orders (
    order_id    TEXT PRIMARY KEY,
    customer_id TEXT REFERENCES customers(customer_id),
    order_date  TEXT,
    status      TEXT CHECK(status IN ('entregado','enviado','cancelado','pendiente'))
);

CREATE TABLE order_items (
    item_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id   TEXT REFERENCES orders(order_id),
    product_id TEXT REFERENCES products(product_id),
    quantity   INTEGER NOT NULL,
    unit_price REAL NOT NULL,
    discount   REAL DEFAULT 0.0
);
''')
conn.commit()
print('Tablas creadas.')
```

```
Tablas creadas.
```

```
# Insertar datos de ejemplo
customers_data = [
    ('C001','Ana García','ana.garcia@email.com','Bogotá','Colombia'),
    ('C002','Carlos López','carlos.lopez@email.com','Medellín','Colombia'),
    ('C003','María Rodríguez','maria.rodriguez@email.com','Cali','Colombia'),
    ('C004','Pedro Martínez','pedro.martinez@email.com','Barranquilla','Colombia'),
    ('C005','Laura Sánchez','laura.sanchez@email.com','Cartagena','Colombia'),
    ('C006','Juan Torres','juan.torres@email.com','Bogotá','Colombia'),
    ('C007','Sofía Herrera','sofia.herrera@email.com','Medellín','Colombia'),
    ('C008','Diego Flores','diego.flores@email.com','Cali','Colombia'),
    ('C009','Valentina Cruz','valentina.cruz@email.com','Bucaramanga','Colombia'),
    ('C010','Andrés Jiménez','andres.jimenez@email.com','Bogotá','Colombia'),
]

products_data = [
    ('P101','Laptop HP 15','Electrónica',1200.00,15),
    ('P205','Auriculares Sony WH','Electrónica',150.00,40),
    ('P310','Silla Ergonómica','Muebles',350.00,20),
    ('P410','Mouse Inalámbrico','Electrónica',45.00,100),
    ('P501','Libro Python Data Science','Libros',35.00,60),
    ('P602','Teclado Mecánico','Electrónica',120.00,30),
    ('P710','Cámara Web HD','Electrónica',80.00,25),
    ('P801','Monitor 24 pulgadas','Electrónica',280.00,18),
    ('P901','Escritorio de Pie','Muebles',500.00,10),
]

orders_data = [
    ('ORD-001','C001','2024-01-05','entregado'),
    ('ORD-002','C002','2024-01-06','entregado'),
    ('ORD-003','C003','2024-01-07','enviado'),
    ('ORD-004','C001','2024-01-08','entregado'),
    ('ORD-005','C004','2024-01-10','entregado'),
    ('ORD-006','C005','2024-01-12','cancelado'),
    ('ORD-007','C006','2024-01-14','entregado'),
    ('ORD-008','C007','2024-01-15','entregado'),
    ('ORD-009','C008','2024-01-16','enviado'),
    ('ORD-010','C002','2024-01-18','entregado'),
    ('ORD-011','C009','2024-01-20','entregado'),
    ('ORD-012','C010','2024-01-22','entregado'),
    ('ORD-013','C003','2024-02-01','entregado'),
    ('ORD-014','C001','2024-02-05','entregado'),
```

```
    ('ORD-015','C004','2024-02-08','cancelado'),
    ('ORD-016','C002','2024-02-10','entregado'),
    ('ORD-017','C005','2024-02-12','entregado'),
    ('ORD-018','C006','2024-02-16','entregado'),
    ('ORD-019','C007','2024-02-20','entregado'),
    ('ORD-020','C010','2024-03-12','entregado'),
]

order_items_data = [
    ('ORD-001','P101',1,1200.00,0.10),
    ('ORD-002','P205',2,150.00,0.00),
    ('ORD-003','P310',1,350.00,0.05),
    ('ORD-004','P410',1,45.00,0.00),
    ('ORD-005','P501',3,35.00,0.15),
    ('ORD-006','P101',1,1200.00,0.10),
    ('ORD-007','P602',1,120.00,0.00),
    ('ORD-008','P710',2,80.00,0.05),
    ('ORD-009','P310',2,350.00,0.10),
    ('ORD-010','P801',1,280.00,0.00),
    ('ORD-011','P501',1,35.00,0.00),
    ('ORD-012','P410',3,45.00,0.00),
    ('ORD-013','P602',1,120.00,0.10),
    ('ORD-014','P901',1,500.00,0.05),
    ('ORD-015','P710',1,80.00,0.00),
    ('ORD-016','P801',2,280.00,0.10),
    ('ORD-017','P310',1,350.00,0.00),
    ('ORD-018','P901',1,500.00,0.00),
    ('ORD-019','P101',1,1200.00,0.10),
    ('ORD-020','P901',1,500.00,0.05),
]

cursor.executemany('INSERT INTO customers VALUES (?,?,?,?,?)', customers_data)
cursor.executemany('INSERT INTO products VALUES (?,?,?,?,?)', products_data)
cursor.executemany('INSERT INTO orders VALUES (?,?,?,?)', orders_data)
cursor.executemany('INSERT INTO order_items(order_id,product_id,quantity,unit_price,discount) VALUES (?,?,?,?,?)', order_items_
conn.commit()
print('Datos insertados correctamente.')
```

```
Datos insertados correctamente.
```

2. SELECT básico y filtros (WHERE)

```
# Seleccionar todas las columnas de una tabla
query('SELECT * FROM customers')
```

	customer_id	name	email	city	country
0	C001	Ana García	ana.garcia@email.com	Bogotá	Colombia
1	C002	Carlos López	carlos.lopez@email.com	Medellín	Colombia
2	C003	María Rodríguez	maria.rodriguez@email.com	Cali	Colombia
3	C004	Pedro Martínez	pedro.martinez@email.com	Barranquilla	Colombia
4	C005	Laura Sánchez	laura.sanchez@email.com	Cartagena	Colombia
5	C006	Juan Torres	juan.torres@email.com	Bogotá	Colombia
6	C007	Sofía Herrera	sofia.herrera@email.com	Medellín	Colombia
7	C008	Diego Flores	diego.flores@email.com	Cali	Colombia
8	C009	Valentina Cruz	valentina.cruz@email.com	Bucaramanga	Colombia
9	C010	Andrés Jiménez	andres.jimenez@email.com	Bogotá	Colombia

```
# Seleccionar columnas específicas
query('SELECT name, city FROM customers')
```

	name	city
0	Ana García	Bogotá
1	Carlos López	Medellín
2	María Rodríguez	Cali
3	Pedro Martínez	Barranquilla
4	Laura Sánchez	Cartagena
5	Juan Torres	Bogotá
6	Sofía Herrera	Medellín
7	Diego Flores	Cali
8	Valentina Cruz	Bucaramanga
9	Andrés Jiménez	Bogotá

```
# Filtrar con WHERE
query("SELECT * FROM customers WHERE city = 'Bogotá'")
```

	customer_id	name	email	city	country
0	C001	Ana García	ana.garcia@email.com	Bogotá	Colombia
1	C006	Juan Torres	juan.torres@email.com	Bogotá	Colombia
2	C010	Andrés Jiménez	andres.jimenez@email.com	Bogotá	Colombia

```
# Múltiples condiciones con AND / OR
query("SELECT * FROM products WHERE category = 'Electrónica' AND price < 200")
```

	product_id	name	category	price	stock
0	P205	Auriculares Sony WH	Electrónica	150.0	40
1	P410	Mouse Inalámbrico	Electrónica	45.0	100
2	P602	Teclado Mecánico	Electrónica	120.0	30
3	P710	Cámara Web HD	Electrónica	80.0	25

```
# Operador IN
query("SELECT * FROM orders WHERE status IN ('cancelado', 'enviado')")
```

	order_id	customer_id	order_date	status
0	ORD-003	C003	2024-01-07	enviado
1	ORD-006	C005	2024-01-12	cancelado
2	ORD-009	C008	2024-01-16	enviado
3	ORD-015	C004	2024-02-08	cancelado

```
# Operador LIKE (buscar patrones en texto)
query("SELECT * FROM customers WHERE name LIKE 'A%'")
```

	customer_id	name	email	city	country
0	C001	Ana García	ana.garcia@email.com	Bogotá	Colombia
1	C010	Andrés Jiménez	andres.jimenez@email.com	Bogotá	Colombia

```
# Operador BETWEEN
query("SELECT * FROM products WHERE price BETWEEN 100 AND 400")
```

	product_id	name	category	price	stock
0	P205	Auriculares Sony WH	Electrónica	150.0	40
1	P310	Silla Ergonómica	Muebles	350.0	20
2	P602	Teclado Mecánico	Electrónica	120.0	30
3	P801	Monitor 24 pulgadas	Electrónica	280.0	18

Ejercicios — Bloque 2

Escribe la consulta SQL para cada uno de los siguientes ejercicios.

E2.1 — Lista todos los clientes de Medellín o Cali.

```
# Tu respuesta aquí
query("""
SELECT *
FROM customers
WHERE city IN ('Medellín', 'Cali')
""")
```

	customer_id	name	email	city	country
0	C002	Carlos López	carlos.lopez@email.com	Medellín	Colombia
1	C003	María Rodríguez	maria.rodriguez@email.com	Cali	Colombia
2	C007	Sofía Herrera	sofia.herrera@email.com	Medellín	Colombia
3	C008	Diego Flores	diego.flores@email.com	Cali	Colombia

E2.2 — Lista los productos de la categoría 'Muebles' con precio mayor a 400.

```
# Tu respuesta aquí
query("""
SELECT *
FROM products
WHERE category = 'Muebles'
  AND price > 400
""")
```

	product_id	name	category	price	stock
0	P901	Escritorio de Pie	Muebles	500.0	10

---

3. Ordenamiento (ORDER BY) y límites (LIMIT)

```
# Ordenar por precio ascendente
query('SELECT name, price FROM products ORDER BY price ASC')
```

	name	price
0	Libro Python Data Science	35.0
1	Mouse Inalámbrico	45.0
2	Cámara Web HD	80.0
3	Teclado Mecánico	120.0
4	Auriculares Sony WH	150.0
5	Monitor 24 pulgadas	280.0
6	Silla Ergonómica	350.0
7	Escritorio de Pie	500.0
8	Laptop HP 15	1200.0

```
# Top 3 productos más caros
query('SELECT name, price FROM products ORDER BY price DESC LIMIT 3')
```

	name	price
0	Laptop HP 15	1200.0
1	Escritorio de Pie	500.0
2	Silla Ergonómica	350.0

```
# Ordenar por múltiples columnas
query('SELECT name, category, price FROM products ORDER BY category ASC, price DESC')
```

	name	category	price
0	Laptop HP 15	Electrónica	1200.0
1	Monitor 24 pulgadas	Electrónica	280.0
2	Auriculares Sony WH	Electrónica	150.0
3	Teclado Mecánico	Electrónica	120.0
4	Cámara Web HD	Electrónica	80.0
5	Mouse Inalámbrico	Electrónica	45.0
6	Libro Python Data Science	Libros	35.0
7	Escritorio de Pie	Muebles	500.0
8	Silla Ergonómica	Muebles	350.0

4. Funciones de agregación

```
# COUNT — contar registros
query('SELECT COUNT(*) AS total_ordenes FROM orders')
```

	total_ordenes
0	20

```
# SUM — sumar valores
query('SELECT SUM(quantity * unit_price) AS ingresos_brutos FROM order_items')
```

	ingresos_brutos
0	8440.0

```
# AVG, MAX, MIN
query('''
SELECT
    AVG(price) AS precio_promedio,
    MAX(price) AS precio_maximo,
    MIN(price) AS precio_minimo
FROM products
''')
```

	precio_promedio	precio_maximo	precio_minimo
0	306.666667	1200.0	35.0

```
# Calcular ingresos netos (con descuento)
query('''
SELECT
    ROUND(SUM(quantity * unit_price), 2)                    AS ingresos_brutos,
    ROUND(SUM(quantity * unit_price * discount), 2)         AS total_descuentos,
    ROUND(SUM(quantity * unit_price * (1 - discount)), 2)   AS ingresos_netos
FROM order_items
''')
```

	ingresos_brutos	total_descuentos	ingresos_netos
0	8440.0	589.25	7850.75

5. Agrupación (GROUP BY) y HAVING

```
# Ventas por categoría de producto
query('''
SELECT
    p.category,
    COUNT(oi.item_id)                                        AS cantidad_items,
    ROUND(SUM(oi.quantity * oi.unit_price * (1-oi.discount)), 2) AS ingresos_netos
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY ingresos_netos DESC
''')
```

	category	cantidad_items	ingresos_netos
0	Electrónica	12	4964.00
1	Muebles	6	2762.50
2	Libros	2	124.25

```
# Clientes con más de 1 pedido (HAVING)
query('''
SELECT
    customer_id,
    COUNT(*) AS num_ordenes
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY num_ordenes DESC
''')
```

	customer_id	num_ordenes
0	C002	3
1	C001	3
2	C010	2
3	C007	2
4	C006	2
5	C005	2
6	C004	2
7	C003	2

```
# Ordenes por mes
query('''
SELECT
    SUBSTR(order_date, 1, 7) AS mes,
    COUNT(*) AS total_ordenes,
    COUNT(CASE WHEN status = 'entregado' THEN 1 END) AS entregadas,
    COUNT(CASE WHEN status = 'cancelado' THEN 1 END) AS canceladas
FROM orders
GROUP BY mes
ORDER BY mes
''')
```

	mes	total_ordenes	entregadas	canceladas
0	2024-01	12	9	1
1	2024-02	7	6	1
2	2024-03	1	1	0

Ejercicios — Bloque 5

E5.1 — ¿Cuántos clientes hay por ciudad? Ordena de mayor a menor.

```
# Tu respuesta aquí
query("""
SELECT city, COUNT(*) AS num_clientes
FROM customers
GROUP BY city
ORDER BY num_clientes DESC
""")
```

	city	num_clientes
0	Bogotá	3
1	Medellín	2
2	Cali	2
3	Cartagena	1
4	Bucaramanga	1
5	Barranquilla	1

E5.2 — Muestra los productos vendidos más de 2 veces (suma de `quantity`).

```
# Tu respuesta aquí
query("""
SELECT p.product_id, p.name, SUM(oi.quantity) AS unidades_vendidas
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.name
HAVING SUM(oi.quantity) > 2
ORDER BY unidades_vendidas DESC
""")
```

	product_id	name	unidades_vendidas
0	P310	Silla Ergonómica	4
1	P410	Mouse Inalámbrico	4
2	P501	Libro Python Data Science	4
3	P101	Laptop HP 15	3
4	P710	Cámara Web HD	3
5	P801	Monitor 24 pulgadas	3
6	P901	Escritorio de Pie	3

---

6. JOINs entre tablas

```
# INNER JOIN — ordenes con nombre del cliente
query('''
SELECT
    o.order_id,
    c.name AS cliente,
    c.city,
    o.order_date,
    o.status
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date
''')
```

	order_id	cliente	city	order_date	status
0	ORD-001	Ana García	Bogotá	2024-01-05	entregado
1	ORD-002	Carlos López	Medellín	2024-01-06	entregado
2	ORD-003	María Rodríguez	Cali	2024-01-07	enviado
3	ORD-004	Ana García	Bogotá	2024-01-08	entregado
4	ORD-005	Pedro Martínez	Barranquilla	2024-01-10	entregado
5	ORD-006	Laura Sánchez	Cartagena	2024-01-12	cancelado
6	ORD-007	Juan Torres	Bogotá	2024-01-14	entregado
7	ORD-008	Sofía Herrera	Medellín	2024-01-15	entregado
8	ORD-009	Diego Flores	Cali	2024-01-16	enviado
9	ORD-010	Carlos López	Medellín	2024-01-18	entregado
10	ORD-011	Valentina Cruz	Bucaramanga	2024-01-20	entregado
11	ORD-012	Andrés Jiménez	Bogotá	2024-01-22	entregado
12	ORD-013	María Rodríguez	Cali	2024-02-01	entregado
13	ORD-014	Ana García	Bogotá	2024-02-05	entregado
14	ORD-015	Pedro Martínez	Barranquilla	2024-02-08	cancelado
15	ORD-016	Carlos López	Medellín	2024-02-10	entregado
16	ORD-017	Laura Sánchez	Cartagena	2024-02-12	entregado
17	ORD-018	Juan Torres	Bogotá	2024-02-16	entregado
18	ORD-019	Sofía Herrera	Medellín	2024-02-20	entregado
19	ORD-020	Andrés Jiménez	Bogotá	2024-03-12	entregado

```
# JOIN de 3 tablas — detalle completo de la orden
query('''
SELECT
    o.order_id,
    c.name       AS cliente,
    p.name       AS producto,
    p.category,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    ROUND(oi.quantity * oi.unit_price * (1-oi.discount), 2) AS total_linea
FROM orders o
JOIN customers c    ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id
ORDER BY o.order_date, o.order_id
''')
```

	order_id	cliente	producto	category	quantity	unit_price	discount	total_linea
0	ORD-001	Ana García	Laptop HP 15	Electrónica	1	1200.0	0.10	1080.00
1	ORD-002	Carlos López	Auriculares Sony WH	Electrónica	2	150.0	0.00	300.00
2	ORD-003	María Rodríguez	Silla Ergonómica	Muebles	1	350.0	0.05	332.50
3	ORD-004	Ana García	Mouse Inalámbrico	Electrónica	1	45.0	0.00	45.00
4	ORD-005	Pedro Martínez	Libro Python Data Science	Libros	3	35.0	0.15	89.25
5	ORD-006	Laura Sánchez	Laptop HP 15	Electrónica	1	1200.0	0.10	1080.00
6	ORD-007	Juan Torres	Teclado Mecánico	Electrónica	1	120.0	0.00	120.00
7	ORD-008	Sofía Herrera	Cámara Web HD	Electrónica	2	80.0	0.05	152.00
8	ORD-009	Diego Flores	Silla Ergonómica	Muebles	2	350.0	0.10	630.00
9	ORD-010	Carlos López	Monitor 24 pulgadas	Electrónica	1	280.0	0.00	280.00
10	ORD-011	Valentina Cruz	Libro Python Data Science	Libros	1	35.0	0.00	35.00
11	ORD-012	Andrés Jiménez	Mouse Inalámbrico	Electrónica	3	45.0	0.00	135.00
12	ORD-013	María Rodríguez	Teclado Mecánico	Electrónica	1	120.0	0.10	108.00
13	ORD-014	Ana García	Escritorio de Pie	Muebles	1	500.0	0.05	475.00
14	ORD-015	Pedro Martínez	Cámara Web HD	Electrónica	1	80.0	0.00	80.00
15	ORD-016	Carlos López	Monitor 24 pulgadas	Electrónica	2	280.0	0.10	504.00
16	ORD-017	Laura Sánchez	Silla Ergonómica	Muebles	1	350.0	0.00	350.00
17	ORD-018	Juan Torres	Escritorio de Pie	Muebles	1	500.0	0.00	500.00
18	ORD-019	Sofía Herrera	Laptop HP 15	Electrónica	1	1200.0	0.10	1080.00
19	ORD-020	Andrés Jiménez	Escritorio de Pie	Muebles	1	500.0	0.05	475.00

```
# LEFT JOIN — clientes que NO tienen ordenes
# (En este dataset todos tienen, pero el patrón es importante)
query('''
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS total_ordenes
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_ordenes DESC
''')
```

	customer_id	name	total_ordenes
0	C001	Ana García	3
1	C002	Carlos López	3
2	C003	María Rodríguez	2
3	C004	Pedro Martínez	2
4	C005	Laura Sánchez	2
5	C006	Juan Torres	2
6	C007	Sofía Herrera	2
7	C010	Andrés Jiménez	2
8	C008	Diego Flores	1
9	C009	Valentina Cruz	1

Ejercicios — Bloque 6

E6.1 — Lista las órdenes entregadas con el nombre del cliente y el total de la orden.

```
# Tu respuesta aquí
query("""
```

```
SELECT
    o.order_id,
    c.name AS cliente,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS total_orden
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status = 'entregado'
GROUP BY o.order_id, c.name
ORDER BY total_orden DESC
""")
```

	order_id	cliente	total_orden
0	ORD-001	Ana García	1080.00
1	ORD-019	Sofía Herrera	1080.00
2	ORD-016	Carlos López	504.00
3	ORD-018	Juan Torres	500.00
4	ORD-014	Ana García	475.00
5	ORD-020	Andrés Jiménez	475.00
6	ORD-017	Laura Sánchez	350.00
7	ORD-002	Carlos López	300.00
8	ORD-010	Carlos López	280.00
9	ORD-008	Sofía Herrera	152.00
10	ORD-012	Andrés Jiménez	135.00
11	ORD-007	Juan Torres	120.00
12	ORD-013	María Rodríguez	108.00
13	ORD-005	Pedro Martínez	89.25
14	ORD-004	Ana García	45.00
15	ORD-011	Valentina Cruz	35.00

E6.2 — Para cada producto, muestra cuántas veces fue comprado y el ingreso total que generó.

```
# Tu respuesta aquí
query("""
SELECT
    p.product_id,
    p.name,
    COUNT(oi.item_id) AS veces_comprado,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS ingreso_total
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY ingreso_total DESC
""")
```

	product_id	name	veces_comprado	ingreso_total
0	P101	Laptop HP 15	3	3240.00
1	P901	Escritorio de Pie	3	1450.00
2	P310	Silla Ergonómica	3	1312.50
3	P801	Monitor 24 pulgadas	2	784.00
4	P205	Auriculares Sony WH	1	300.00
5	P710	Cámara Web HD	2	232.00
6	P602	Teclado Mecánico	2	228.00
7	P410	Mouse Inalámbrico	2	180.00
8	P501	Libro Python Data Science	2	124.25

7. Subconsultas

```
# Productos con precio mayor al promedio
query('''
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC
''')
```

	name	price
0	Laptop HP 15	1200.0
1	Escritorio de Pie	500.0
2	Silla Ergonómica	350.0

```
# Clientes que hicieron al menos una orden cancelada
query('''
SELECT name, email, city
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id
    FROM orders
    WHERE status = 'cancelado'
)
''')
```

	name	email	city
0	Pedro Martínez	pedro.martinez@email.com	Barranquilla
1	Laura Sánchez	laura.sanchez@email.com	Cartagena

```
# Subconsulta en FROM (tabla derivada)
query('''
SELECT
    cliente,
    total_gastado,
    CASE
        WHEN total_gastado >= 2000 THEN 'VIP'
        WHEN total_gastado >= 500  THEN 'Regular'
        ELSE 'Ocasional'
    END AS segmento
FROM (
    SELECT
        c.name AS cliente,
        ROUND(SUM(oi.quantity * oi.unit_price * (1-oi.discount)), 2) AS total_gastado
    FROM customers c
    JOIN orders o     ON c.customer_id = o.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.status = 'entregado'
    GROUP BY c.customer_id
) sub
ORDER BY total_gastado DESC
''')
```

	cliente	total_gastado	segmento
0	Ana García	1600.00	Regular
1	Sofía Herrera	1232.00	Regular
2	Carlos López	1084.00	Regular
3	Juan Torres	620.00	Regular
4	Andrés Jiménez	610.00	Regular
5	Laura Sánchez	350.00	Ocasional
6	María Rodríguez	108.00	Ocasional
7	Pedro Martínez	89.25	Ocasional
8	Valentina Cruz	35.00	Ocasional

8. Funciones de texto y fecha

```
# Funciones de texto
query('''
SELECT
    name,
    UPPER(name)            AS nombre_mayus,
    LENGTH(name)           AS longitud,
    SUBSTR(name, 1, 5)     AS primeros_5,
    REPLACE(email, '@email.com', '') AS usuario_email
FROM customers
LIMIT 5
''')
```

	name	nombre_mayus	longitud	primeros_5	usuario_email
0	Ana García	ANA GARCíA	10	Ana G	ana.garcia
1	Carlos López	CARLOS LóPEZ	12	Carlo	carlos.lopez
2	María Rodríguez	MARíA RODRíGUEZ	15	María	maria.rodriguez
3	Pedro Martínez	PEDRO MARTíNEZ	14	Pedro	pedro.martinez
4	Laura Sánchez	LAURA SáNCHEZ	13	Laura	laura.sanchez

```
# Funciones de fecha en SQLite
query('''
SELECT
    order_id,
    order_date,
    SUBSTR(order_date, 1, 4) AS anio,
    SUBSTR(order_date, 6, 2) AS mes,
    SUBSTR(order_date, 9, 2) AS dia
FROM orders
LIMIT 5
''')
```

	order_id	order_date	anio	mes	dia
0	ORD-001	2024-01-05	2024	01	05
1	ORD-002	2024-01-06	2024	01	06
2	ORD-003	2024-01-07	2024	01	07
3	ORD-004	2024-01-08	2024	01	08
4	ORD-005	2024-01-10	2024	01	10

9. Ejercicios integradores

EI.1 — Ranking de los 5 clientes con mayor gasto total en órdenes entregadas, mostrando: nombre, ciudad, número de órdenes y total gastado.

```
# Tu respuesta aquí
query("""
SELECT
    c.name,
    c.city,
    COUNT(DISTINCT o.order_id) AS num_ordenes,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS total_gastado
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status = 'entregado'
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_gastado DESC
LIMIT 5
""")
```

	name	city	num_ordenes	total_gastado
0	Ana García	Bogotá	3	1600.0
1	Sofía Herrera	Medellín	2	1232.0
2	Carlos López	Medellín	3	1084.0
3	Juan Torres	Bogotá	2	620.0
4	Andrés Jiménez	Bogotá	2	610.0

EI.2 — ¿Cuál es el producto más vendido (en unidades) por categoría?

```
# Tu respuesta aquí
query("""
WITH ventas AS (
    SELECT
        p.category,
        p.product_id,
        p.name,
        SUM(oi.quantity) AS unidades_vendidas
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.category, p.product_id, p.name
)
SELECT category, name, unidades_vendidas
FROM (
    SELECT
        category,
        name,
        unidades_vendidas,
        ROW_NUMBER() OVER (PARTITION BY category ORDER BY unidades_vendidas DESC) AS rn
    FROM ventas
) t
WHERE rn = 1
ORDER BY category
""")
```

	category	name	unidades_vendidas
0	Electrónica	Mouse Inalámbrico	4
1	Libros	Libro Python Data Science	4
2	Muebles	Silla Ergonómica	4

EI.3 — Tasa de cancelación por mes: muestra el porcentaje de órdenes canceladas sobre el total de órdenes, por mes.

```
# Tu respuesta aquí
query("""
SELECT
    SUBSTR(order_date, 1, 7) AS mes,
    COUNT(*) AS total_ordenes,
    SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) AS ordenes_canceladas,
    ROUND(
        100.0 * SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) / COUNT(*),
        2
```

```
    ) AS tasa_cancelacion_pct
FROM orders
GROUP BY mes
ORDER BY mes
```

	mes	total_ordenes	ordenes_canceladas	tasa_cancelacion_pct
0	2024-01	12	1	8.33
1	2024-02	7	1	14.29
2	2024-03	1	0	0.00

EI.4 — Lista los productos que nunca fueron vendidos (usa LEFT JOIN o NOT IN).

```
# Tu respuesta aquí
query("""
SELECT p.product_id, p.name, p.category
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL
ORDER BY p.name
""")
```

No entries Filter

index: to
product_id: to
name: to
category: to
Search by all fields:

index	product_id	name	category

Show 10 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

Soluciones de referencia

Descomenta y ejecuta cada celda para ver la solución.